

Neural Networks

Feed Forward Networks and Backpropagation

John Snyder

STAT 9340: Spring 2023

Section 1

Shallow Neural Networks

Review of Neural Networks

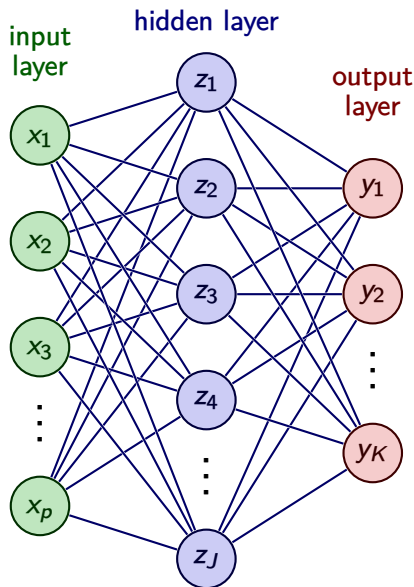
We begin with a discussion on a “shallow” feed forward neural network.

- These models include a hidden layer containing variables constructed from nonlinear transformations of a combination of the input variables. The output layer is then similarly computed as a linear combination of the variables (neurons, units) in the hidden layer.
- The sequence of (simple) linear combinations and nonlinear transformations that happen from input to output are what allow these models to fit very complex response surfaces.

These networks are called “feed forward” because they progress from input to output through a composition of the hidden layers.

- No recurrence, like we will see with some models later

Neural network notation and image



The "multilayer perceptron" has inputs, hidden layers, and outputs

- The inputs x_i are our covariates
- The hidden units h_j are perceptrons (nodes)
 - ▶ **EACH** receiving weighted input from **every** x_i as well as bias
- The outputs y_k are also nodes that produce the "response(s)"
 - ▶ $K = 1$ for single output regressions
 - ▶ $K > 1$ for classification problems
 - ★ Each output is a probability of class membership

Inputs into Hidden Layer

The idea with each hidden neuron in a network is to transform a linear combination of inputs via a nonlinear *activation function*. We have

- 1 $\mathbf{x}$, a p -dimensional vector of inputs.
- 2 $\mathbf{y}$, a K -dimensional vector of outputs.
- 3 For each of the hidden units z_j , $j = 1, \dots, J$, we have weights w_{ji} , $i = 0, \dots, p$ which map the inputs to the hidden units.
 - Simple linear combination with w_{j0} being called the *bias* (like an intercept).
- 4 The nonlinear *activation function* f .

We then have

$$z_j = f \left(\sum_{i=0}^p w_{ji} x_i \right).$$

With $x_0 = 1$ to simplify notation and include the bias.

Hidden Layer to Output

We then similarly process the hidden units z_j with new weights β_{kj} and activate with g to arrive at the output y_k as

$$y_k = g \left(\sum_{j=0}^J \beta_{kj} z_j \right), \quad k = 1, \dots, K$$

again with $z_0 = 1$ to simplify notation for the bias term.

- Because this is the output layer, the function g is typically different than the activation functions f .
 - ▶ This is the case for deeper models as well
- $g(x) = x$ (identity) for regression
- $g(x_i) = \frac{\exp(x_i)}{\sum_i \exp(x_i)}$ (softmax) for classification

Summary

In summary, to compute the output y_k from inputs in a 1-hidden layer feed-forward neural network, we can write

$$y_k(\mathbf{x}; \boldsymbol{\beta}, \mathbf{w}) = g \left(\sum_{j=0}^J \beta_{kj} \cdot f \left(\sum_{i=0}^p w_{ji} x_i \right) \right).$$

- This model has $J(p + 1) + K(J + 1)$ parameters
- f being a nonlinear function induces nonlinearity in the model
 - ▶ Without this we just have a big linear regression.

Activation Functions: Sigmoid-y Transformations

Sigmoid:

$$f(z) = \frac{1}{1 + \exp(-z)}$$

- Asymptotes at 0 and 1.
- Convenient because $f'(z) = f(z)(1 - f(z))$, eases calculation.

Hyperbolic Tangent:

$$f(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

- Asymptotes at -1 and 1.

The key advantage of these activation functions is activated units will always be “clamped” to be within the asymptotes, however very large/small inputs can lead to “vanishing gradients” with large models.

Activation functions: ReLU

Rectified Linear Unit(ReLU):

$$f(z) = \max(0, z)$$

- Truncates negative values to 0.
- When many hidden units with ReLU activations combine, we end up with folded hyperplanes in the response surface.
- Most often used for neural networks because it's truncation forces sparsity and the gradient is simple(so trains faster).
- Major advantage is it helps with vanishing gradients from repeated chain rule applications.

Leaky ReLU

$$f(z) = \max(0, z) - \alpha \max(0, -z) \quad \text{for some } \alpha \geq 0$$

- Allows some negative contribution (controlled by α) to contribute
- This *can* enhance flexibility by not allowing neurons to “die.”

Activation functions: Swish

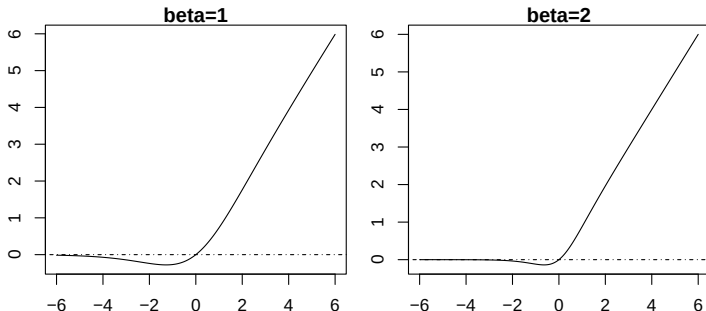
One other interesting transformation is the Swish function defined by

$$f(z) = z\sigma(\beta z) \quad \text{for some } \beta \geq 0$$

where $\beta \geq 0$ is a hyperparameter and $\sigma(\cdot)$ is the sigmoid function.

- Enhances sparsity like ReLU, while being differentiable near 0.

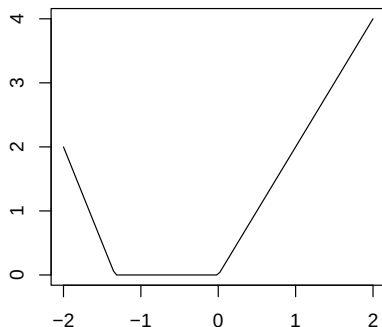
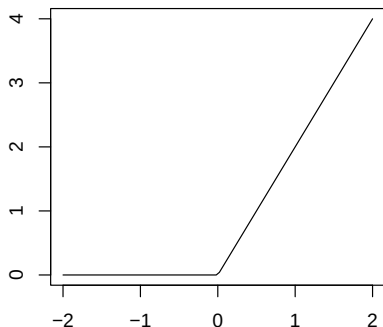
```
mySwish <- function(z, Beta) z*(1/(1+exp(-Beta*z)))  
z <- seq(-6,6,length.out=1000)  
plot(x = z, y = mySwish(z, Beta=1),type = 'l',main='beta=1'); abline(h=0, lty=4)  
plot(x = z, y = mySwish(z, Beta=2),type = 'l',main='beta=2'); abline(h=0, lty=4)
```



How activation functions combine together

Linear combinations of the simple ReLU function create a piecewise linear representation of a function

```
My_ReLU <- function(x) ifelse(x > 0, x, 0)
# ReLU functions
x <- seq(-2,2,length.out=100)
# z_1
y1 <- My_ReLU(2*x)
# z_1 + z_2
y2 <- My_ReLU(2*x) + My_ReLU(-3*x-4)
plot(x,y1,type="l")
plot(x,y2,type="l")
```

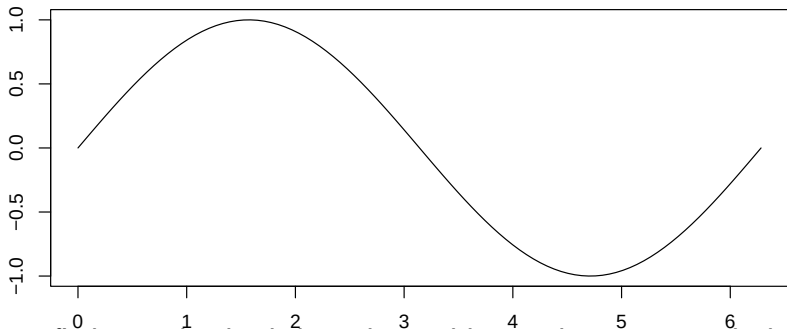


Approximating $\sin(x)$ with ReLU activations

Say we wish to approximate

$$y = \sin(x)$$

```
x <- seq(0, 2*pi, length.out= 100)
sin_x <- sin(x)
plot(x,sin_x, type="l")
```

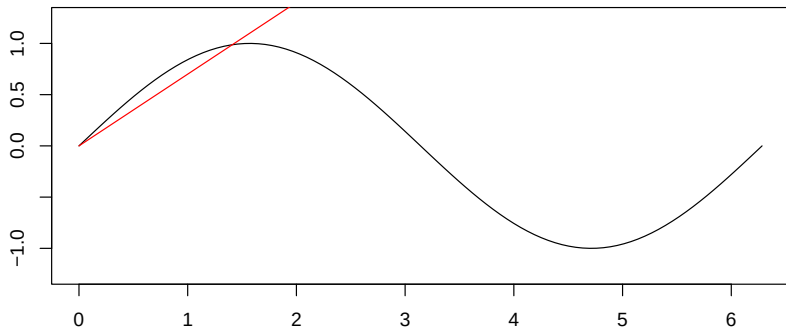


We can find some hardcoded weights and biases that give straight line approximations.

Approximating $\sin(x)$ with ReLU activations 1/3

```
x <- seq(0, 2*pi, length.out= 100)
sin_x <- sin(x)
plot(x, sin_x, type="l", ylim=c(-1.25, 1.25))

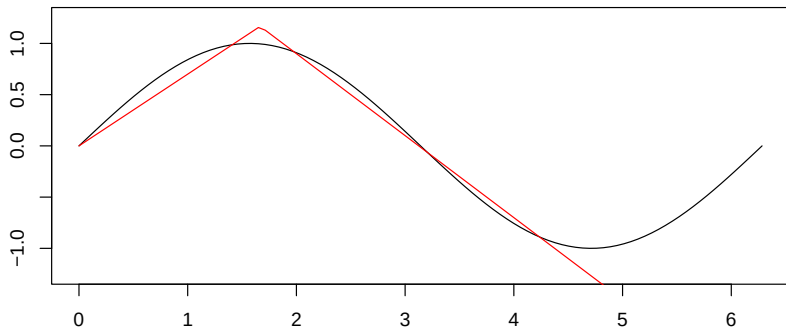
# y = 0 + 1*z_1
SumOfReLUs <- My_ReLU(.7*x)
lines(x, SumOfReLUs, col="red")
```



Approximating $\sin(x)$ with ReLU activations 2/3

```
x <- seq(0, 2*pi, length.out= 100)
sin_x <- sin(x)
plot(x, sin_x, type="l", ylim=c(-1.25, 1.25))

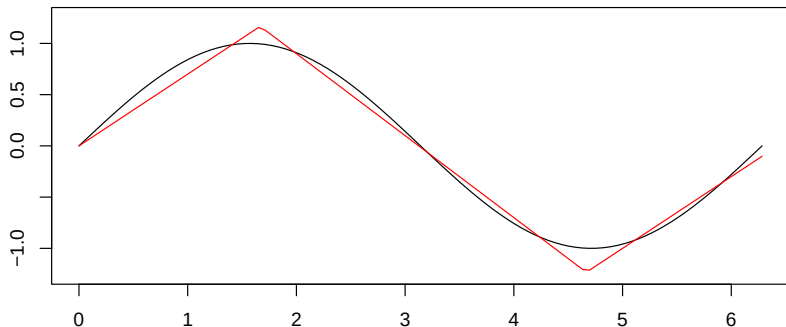
#      y      = 0 + 1*z_1      +      -1*z_2
SumOfReLUs <- My_ReLU(.7*x) - My_ReLU(1.5*x-2.5)
lines(x,SumOfReLUs, col="red")
```



Approximating $\sin(x)$ with ReLU activations 3/3

```
x <- seq(0, 2*pi, length.out= 100)
sin_x <- sin(x)
plot(x,sin_x, type="l", ylim=c(-1.25, 1.25))

#      y      = 0 + 1*z_1      +      -1*z_2      +      1*z_3
SumOfReLUs <- My_ReLU(.7*x) - My_ReLU(1.5*x-2.5) + My_ReLU(1.5*x-7)
lines(x,SumOfReLUs, col="red")
```



Model fitting (backpropagation)

We select an objective function $Q(\mathbf{w}, \beta; \mathbf{x}, \tilde{\mathbf{y}})$ (e.g., squared error, cross-entropy) and then seek a gradient-based approach to obtain “optimal” parameter estimates of the w and β parameters.

The essence of backpropagation is simply the chain rule applied to the gradient for each of the parameters:

$$\frac{\partial Q}{\partial w_{ji}} = \frac{\partial Q}{\partial y_k} \frac{\partial y_k}{\partial z_j} \frac{\partial z_j}{\partial w_{ji}}, \quad j = 1, \dots, J; i = 0, \dots, p,$$
$$\frac{\partial Q}{\partial \beta_{kj}} = \frac{\partial Q}{\partial y_k} \frac{\partial y_k}{\partial \beta_{kj}}, \quad k = 1, \dots, K; j = 0, \dots, J.$$

Backpropagation in a single output FNN

Consider a single output regression problem where we have output/input pairs $\{\tilde{y}_\ell, \mathbf{x}_\ell\}$. We will consider the identity function for the output and a sigmoid function $f(\cdot)$ as the activation. We then have

$$y_\ell = \sum_{j=1}^J \beta_j z_{j\ell}$$
$$z_{j\ell} = f\left(\sum_{i=0}^P w_{ji} x_{i\ell}\right)$$

With the weights β_j and w_{ji} , this completely describes the flow of information through the FNN. To evaluate the performance of the predictions y_ℓ , consider the squared-error objective function:

$$Q(\mathbf{w}, \beta, \{\mathbf{x}, \tilde{\mathbf{y}}\}) = \frac{1}{2} \sum_{\ell} (\tilde{y}_\ell - y_\ell)^2,$$

which we will seek to minimize.

Backpropagation in a single output FNN (cont)

Obtaining the gradients involves us applying the chain rule, respecting the flow of information through the network as we looked at in reverse mode autodiff.

$$\begin{aligned}\frac{\partial Q}{\partial \beta_j} &= \frac{\partial Q}{\partial y_k} \frac{\partial y_k}{\partial \beta_j} = - \sum_{\ell} (\tilde{y}_{\ell} - y_{\ell}) z_{j\ell}, \\ \frac{\partial Q}{\partial w_{ji}} &= \sum_{\ell} \frac{\partial Q}{\partial y} \frac{\partial y}{\partial z_j} \frac{\partial z_j}{\partial w_{ji}} \\ &= \sum_{\ell} \{-(\tilde{y}_{\ell} - y_{\ell})\} \{\beta_j\} \{z_{j\ell} (1 - z_{j\ell}) x_{i\ell}\}.\end{aligned}$$

The $(\tilde{y}_{\ell} - y_{\ell})$ term describes the error in the top level and the terms before the $x_{i\ell}$ in $\frac{\partial Q}{\partial w_{ji}}$ behave like an error term at the hidden layer.

SGD updates for the single output FNN (cont)

The SGD algorithm then considers the following parameter updates for iteration counter r :

$$\beta_j^{(r+1)} = \beta_j^{(r)} - \gamma^{(r)} \frac{1}{L} \sum_{\ell} \frac{\partial Q}{\partial \beta_j},$$
$$w_{ji}^{(r+1)} = w_{ji}^{(r)} - \gamma^{(r)} \frac{1}{L} \sum_{\ell} \frac{\partial Q}{\partial w_{ji}},$$

where $\gamma^{(r)}$ is the learning rate and L is the # of samples in the batch

Updates are done with two steps:

- 1 Forward pass: The current parameter values are fixed at the r th values and $z_{j\ell}$ and y_{ℓ} are calculated.
- 2 Backward pass: Compute the gradients $\frac{\partial Q}{\partial \beta_j}$ and $\frac{\partial Q}{\partial w_{ji}}$ and update parameters using SGD above.

In practice, the x 's are typically scaled and starting values for the w and β parameters can be drawn randomly from a uniform, e.g., $Unif(-a, a)$.

Multiple output regression

It is straightforward to consider multiple output units in the one-hidden layer nonlinear regression model. In this case we write the output $y_{k\ell}$ as

$$y_{k\ell} = \sum_{j=0}^J \beta_{kj} z_{j\ell}, \quad k = 1, \dots, K.$$

Then, the objective function must include an extra summation term,

$$Q(\cdot) = \frac{1}{2} \sum_{\ell} \sum_k (\tilde{y}_{k\ell} - y_{k\ell})^2,$$

and the gradients are given by:

$$\begin{aligned} \frac{\partial Q}{\partial \beta_{kj}} &= - \sum_{\ell} (\tilde{y}_{k\ell} - y_{k\ell}) z_{j\ell}, \\ \frac{\partial Q}{\partial w_{ji}} &= - \sum_{\ell} \left[\sum_k (\tilde{y}_{k\ell} - y_{k\ell}) \beta_{kj} \right] z_{j\ell} (1 - z_{j\ell}) x_{i\ell}. \end{aligned}$$

Full FNN forward and backprop algorithm

Require: Learning rate γ

Init: Initial parameters $\mathbf{w}, \beta$ from Unif $(-a, a)$ (a small)

```
1 while Stopping criterion not met do
2   Sample a minibatch of  $m$  examples from the training set  $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$  with
   corresponding targets  $\{\mathbf{y}_1, \dots, \mathbf{y}_m\}$ ;
3   for  $j \leftarrow 1$  to  $J$  do
4     Calculate  $z_{j\ell}$ ;
5   end
6   for  $k \leftarrow 1$  to  $K$  do
7     Calculate  $y_{k\ell}$ ;
8   end
9   Set  $\hat{\mathbf{g}}_{w_j} = \mathbf{0}, j = 0, \dots, J; \hat{\mathbf{g}}_{\beta_k} = \mathbf{0}, k = 1, \dots, K;$ 
10  for  $\ell \leftarrow 1$  to  $L$  do
11     $\hat{\mathbf{g}}_{\beta_k} \leftarrow \hat{\mathbf{g}}_{\beta_k} + \frac{1}{m} \frac{\partial Q}{\partial \beta_{kj}}, k = 1, \dots, K;$ 
12     $\hat{\mathbf{g}}_{w_j} \leftarrow \hat{\mathbf{g}}_{w_j} + \frac{1}{m} \frac{\partial Q}{\partial w_{ji}}, j = 0, \dots, J;$ 
13  end
14  Apply updates::
15   $\beta_k \leftarrow \beta_k - \gamma \hat{\mathbf{g}}_{\beta_k}; k = 1, \dots, K;$ 
16   $\mathbf{w}_j \leftarrow \mathbf{w}_j - \gamma \hat{\mathbf{g}}_{w_j}; j = 0, \dots, J;$ 
17 end
```

FNN: Two Class Discrimination

If the output is binary, the standard neural net implementation includes one hidden layer and one output unit with a sigmoidal transformation $\sigma(\cdot)$.

Again, assuming $z_{0\ell} = 1$ we have

$$y_\ell = \sigma \left(\sum_{j=0}^J \beta_j z_{j\ell} \right).$$

The hidden units $z_{j\ell}$ are constructed exactly the same as before in the forward pass. We also typically use the cross entropy function

$$Q(\mathbf{w}, \beta, \{\mathbf{x}_\ell, \tilde{y}_\ell\}) = - \sum_{\ell} \tilde{y}_\ell \log y_\ell + (1 - \tilde{y}_\ell) \log (1 - y_\ell)$$

for the loss. Interestingly, the form we have assumed yield the same forms for the gradients:

$$\begin{aligned} \frac{\partial Q}{\partial \beta_j} &= - \sum_{\ell} (\tilde{y}_\ell - y_\ell) z_{j\ell}, \\ \frac{\partial Q}{\partial w_{ji}} &= - \sum_{\ell} (\tilde{y}_\ell - y_\ell) \beta_j z_{j\ell} (1 - z_{j\ell}) x_{i\ell}. \end{aligned}$$

Multi Class Classification

In the case of $K > 2$ category classification, we first consider K output units:

$$o_{kl} = \sum_{j=0}^J \beta_{kj} z_{jl}, \quad k = 1, \dots, K,$$

and the softmax function activates these to probabilities

$$y_{kl} = \frac{\exp(o_{kl})}{\sum_{k'} \exp(o_{k'\ell})}.$$

Again, the objective function is given by cross-entropy

$$Q(\cdot) = - \sum_{\ell} \sum_k \tilde{y}_{k\ell} \log y_{k\ell}.$$

and this again gives the same gradients!

$$\frac{\partial Q}{\partial \beta_{kj}} = - \sum_{\ell} (\tilde{y}_{k\ell} - y_{k\ell}) z_{j\ell},$$

$$\frac{\partial Q}{\partial w_{ji}} = - \sum_{\ell} \left[\sum_k (\tilde{y}_{k\ell} - y_{k\ell}) \beta_{kj} \right] z_{j\ell} (1 - z_{j\ell}) x_{i\ell}.$$

Practical Considerations

One layer NNs do not have many of the issues that deeper models have. Still, there are some considerations.

- **Starting Values:** Usually one can just use Unif ($-.01, .01$). There are some interesting approaches, such as the “Xavier normal initializer” which draws samples from a truncated normal distribution centered on 0 with $\text{stddev} = \sqrt{2 / (\text{fan_in} + \text{fan_out})}$ where fan_in/out are the number of input/output units. Key: Never start at 0, because you will never leave 0.
- **Input standardization:** All inputs should have mean zero and standard deviation one so they are on the same scale.
- **Regularization:** Neural networks tend to overfit. It is critical to include “weight decay,” which is just a regularization. The most common is to consider an L_2 (ridge) penalty to the objective of the form,
 $\lambda J(\beta; \mathbf{w}) = \lambda \left(\sum_{ji} w_{ji}^2 + \sum_{kj} \beta_{kj}^2 \right)$, where $\lambda \geq 0$ is the tuning parameter. This has the effect of adding terms $2\lambda\beta_{kj}$ and $2\lambda w_{ji}$ to the gradient expressions.

Practical Considerations(cont)

- Number of Hidden Units: Probably better to have too many than too few, especially if one considers regularization or ReLU activations which can be self regularizing.
- Multiple Minima: It is not typically possible to ensure your solution is the global minimum, and it isn't really necessary. There are often many “good enough” resting places for the net's weights to converge to.
- Learning rate scheduling: Often we do not want to use a fixed learning rate. We may have the learning rate decrease over time or utilize “warm restarts” to hop out of local minima.
- Stopping Criterion: Usually, we define a fixed number of “epochs” to train the model for. We can also monitor the objective function evaluated on a holdout set of data and stop once the error stops improving in a so called “early stopping” approach.

One layer NN in Tensorflow

```
import tensorflow as tf
import numpy as np

# No relationship!
input_data = np.random.rand(1000, 20)
target_data = np.random.randint(2, size=(1000, 1))

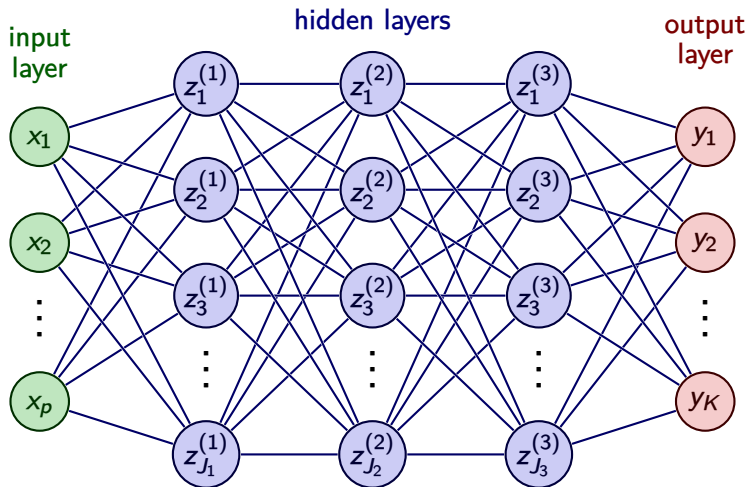
# Input layer
inputs = tf.keras.Input(shape=(input_data.shape[1],))
# Hidden layer
hidden = tf.keras.layers.Dense(units=64, activation='relu')(inputs)
# Output layer
outputs = tf.keras.layers.Dense(units=1, activation='sigmoid')(hidden)
# Define the model
model = tf.keras.Model(inputs=inputs, outputs=outputs)
# Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
# Train the model
model.fit(input_data, target_data, epochs=10, batch_size=32)
```

Section 2

Deep Neural Networks

Deep Neural Networks

To transition to “deep” neural networks, the output from one hidden layer is the input to the next.



The above has $J_1(p + 1) + J_2(J_1 + 1) + J_3(J_2 + 1) + K(J_3 + 1)$ paramters!

Deep Neural Network Formulation

To construct the notation for DNNs, we need to capture the fact that each hidden layer behaves as an input to the *next* hidden layer.

We denote the input layer as layer 0 and the output layer as layer L , giving a $L + 1$ -layer DNN. So the first L layers are written (Note adjusted notation):

$$\mathbf{v}^\ell = g(\mathbf{z}^\ell) = g(\mathbf{W}^\ell \mathbf{v}^{\ell-1} + \mathbf{b}^\ell), \text{ for } 0 < \ell < L,$$

where

- $\mathbf{z}^\ell = \mathbf{W}^\ell \mathbf{v}^{\ell-1} + \mathbf{b}^\ell \in \mathbb{R}^{N_\ell \times 1}$ is the hidden unit value vector
 - ▶ Note: Pre activation!
- $\mathbf{v}^\ell \in \mathbb{R}^{N_\ell \times 1}$ is the activation vector,
- $\mathbf{W}^\ell \in \mathbb{R}^{N_\ell \times N_{\ell-1}}$ the weight matrix
- $\mathbf{b}^\ell \in \mathbb{R}^{N_\ell \times 1}$ the bias vector
- N_ℓ : is the number of hidden units at layer ℓ .

Also

- $\mathbf{v}^0 = \mathbf{x} \in \mathbb{R}^{N_x \times 1}$ are the inputs with dimension N_x
- $g(\cdot) : \mathbb{R}^{N_\ell \times 1} \rightarrow \mathbb{R}^{N_\ell \times 1}$: activation function applied element-wise to $\mathbf{z}^\ell$

Note: Much of this exposition follows from Yu and Deng 2015 Chapter 4

The Output Layer of a DNN

As previously discussed, the “activated” output $\mathbf{v}^L = g(\mathbf{z}^L) = \mathbf{y} \in \mathbb{R}^{N_L}$ depends on the type of data we have.

- Regression: Identity activation, simple linear combination

$$\mathbf{v}^L = \mathbf{z}^L = \mathbf{W}^L \mathbf{v}^{L-1} + \mathbf{b}^L,$$

- Multi-class Classification: Consider classes $\{1, 2, \dots, K\}$. We structure the model so $K = N_L$ and the i th value of v_i^L represents the probability $P(i | \mathbf{x})$, the probability that the input $\mathbf{x}$ belongs to the i th class. The softmax function

$$v_i^L = P(i | \mathbf{x}) = \text{softmax}_i(\mathbf{z}^L) \equiv \frac{\exp(z_i^L)}{\sum_{j=1}^K \exp(z_j^L)}$$

is used to map the linear combination to these probabilities.

Note that the *data* $\tilde{\mathbf{y}} \in \mathbb{R}^{N_L}$ will be an N_L length unit vector with a 1 corresponding to the class of the example.

The forward pass on a Deep Neural Network

Given weights and biases, $\{\mathbf{W}, \mathbf{b}\} = \{\mathbf{W}^\ell, \mathbf{b}^\ell \mid \ell = 1, \dots, L\}$, the forward pass through the network is as follows

Require: Input data $\mathbf{X}$ where columns are observations

Init: Weights $\mathbf{W}$ and biases $\mathbf{b}$

```
1  $\mathbf{V}^0 \leftarrow \mathbf{X};$   
2 for  $\ell \leftarrow 1$  to  $L - 1$  do  
3    $\mathbf{Z}^\ell \leftarrow \mathbf{W}^\ell \mathbf{V}^{\ell-1} + \mathbf{B}^\ell$  where columns of  $\mathbf{B}^\ell$  is  $\mathbf{b}^\ell$  repeated  $M$  times;  
4    $\mathbf{V}^\ell \leftarrow g(\mathbf{Z}^\ell);$   
5 end  
6  $\mathbf{Z}^L \leftarrow \mathbf{W}^L \mathbf{V}^{L-1} + \mathbf{B}^L;$   
7  $\mathbf{V}^L \leftarrow f(\mathbf{Z}^L);$ 
```

$\mathbf{V}^L$ is the output, so the function $f(\cdot)$ is chosen to map to the output data.

- Identity for regression
- Softmax (applied to columns) for categorical data

Setting up backpropagation: Objective functions

For the inputs $m = 1, \dots, M$ we will consider either the MSE or the cross entropy objective function as before:

For regression: we use the MSE

$$Q_{MSE}(\mathbf{W}, \mathbf{b}; \tilde{\mathbf{Y}}, \mathbf{X}) = \frac{1}{M} \sum_{m=1}^M Q_{MSE}(\mathbf{W}, \mathbf{b}; \mathbf{x}_m, \tilde{\mathbf{y}}_m)$$

where

$$Q_{MSE}(\mathbf{W}, \mathbf{b}; \mathbf{x}_m, \tilde{\mathbf{y}}_m) = \frac{1}{2} \left(\mathbf{v}_m^L - \tilde{\mathbf{y}}_m \right)' \left(\mathbf{v}_m^L - \tilde{\mathbf{y}}_m \right)$$

Note that $\tilde{\mathbf{y}}_m$ is boldface to account for the general case of multiple output regression.

Setting up backpropagation: Objective functions

For classification, we use Cross Entropy (or CE)

$$Q_{CE}(\mathbf{W}, \mathbf{b}; \tilde{\mathbf{Y}}, \mathbf{X}) = \frac{1}{M} \sum_{m=1}^M Q_{CE}(\mathbf{W}, \mathbf{b}; \mathbf{x}_m, \tilde{\mathbf{y}}_m)$$

where

$$Q_{CE}(\mathbf{W}, \mathbf{b}; \mathbf{x}_m, \tilde{\mathbf{y}}_m) = - \sum_{k=1}^K \tilde{y}_{km} \log v_{km}^L$$

where $\tilde{y}_k = I_{c=k}$ (note, $I_{c=k} = 1$ if $k = c$ and 0, otherwise). The CE criterion simplifies to $Q = -\log v_c^L$ in this case. In general, $\tilde{y}_k$ can correspond to an observed probability.

Note: Minimizing the CE objective is equivalent to minimizing the Kullback-Leibler divergence (KLD) between the empirical probability distribution $P_{\text{emp}}(k | \mathbf{x})$ and the probability distribution estimated from the DNN $P_{\text{dnn}}(k | \mathbf{x})$

SGD on the DNN

We update the ℓ th layer parameters at the t th update via gradient descent

$$\begin{aligned}\mathbf{W}_{t+1}^\ell &\leftarrow \mathbf{W}_t^\ell - \eta \Delta \mathbf{W}_t^\ell \\ \mathbf{b}_{t+1}^\ell &\leftarrow \mathbf{b}_t^\ell - \eta \Delta \mathbf{b}_t^\ell\end{aligned}$$

The gradient terms are given by:

$$\begin{aligned}\Delta \mathbf{W}_t^\ell &= \frac{1}{M_b} \sum_{m=1}^{M_b} \nabla_{\mathbf{W}_t^\ell} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}_m, \tilde{\mathbf{y}}_m) \\ \Delta \mathbf{b}_t^\ell &= \frac{1}{M_b} \sum_{m=1}^{M_b} \nabla_{\mathbf{b}_t^\ell} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}_m, \tilde{\mathbf{y}}_m)\end{aligned}$$

where $\nabla_{\mathbf{u}} Q(\mathbf{u}, \cdot)$ corresponds of the gradient of Q with respect to $\mathbf{u}$, and we average over M_b minibatch samples.

Top level backpropagation details: Regression

Top Layer: Regression (MSE) formulation

$$\nabla_{\mathbf{W}_t^L} Q_{MSE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) = \nabla_{\mathbf{z}_t^L} Q_{MSE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) \frac{\partial \mathbf{z}_t^L}{\partial \mathbf{W}_t^L}$$

Note,

$$\nabla_{\mathbf{z}_t^L} Q_{MSE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) = \frac{1}{2} \frac{\partial (\mathbf{z}_t^L - \tilde{\mathbf{y}})' (\mathbf{z}_t^L - \tilde{\mathbf{y}})}{\partial \mathbf{z}_t^L} = (\mathbf{v}_t^L - \tilde{\mathbf{y}}) \equiv \mathbf{e}_t^L$$

$$\frac{\partial \mathbf{z}_t^L}{\partial \mathbf{W}_t^L} = \frac{\partial (\mathbf{W}_t^L \mathbf{v}_t^{L-1} + \mathbf{b}_t^L)}{\partial \mathbf{W}_t^L} = (\mathbf{v}_t^{L-1})'$$

So,

$$\nabla_{\mathbf{W}_t^L} Q_{MSE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) = (\mathbf{v}_t^L - \tilde{\mathbf{y}}) (\mathbf{v}_t^{L-1})'$$

Similarly,

$$\nabla_{\mathbf{b}_t^L} Q_{MSE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) = (\mathbf{v}_t^L - \tilde{\mathbf{y}}) \quad \left[\text{recall, } \mathbf{v}_t^L = \mathbf{z}_t^L \text{ in regression case} \right]$$

Top level backpropagation details: Classification

Top Layer: Cross-Entropy (CE) formulation

$$\begin{aligned}\nabla_{\mathbf{W}_t^L} Q_{CE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) &= \nabla_{\mathbf{z}_t^L} Q_{CE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) \frac{\partial \mathbf{z}_t^L}{\partial \mathbf{W}_t^L} \\ &= \mathbf{e}_t^L \frac{\partial \left(\mathbf{W}_t^L \mathbf{v}_t^{L-1} + \mathbf{b}_t^L \right)}{\partial \mathbf{W}_t^L} = \mathbf{e}_t^L \left(\mathbf{v}_t^{L-1} \right)'\end{aligned}$$

where

$$\begin{aligned}\mathbf{e}_t^L &= \nabla_{\mathbf{z}_t^L} Q_{CE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) = - \frac{\partial \sum_{i=1}^K \tilde{y}_i \log \left(\text{softmax}_i \left(\mathbf{z}_t^L \right) \right)}{\partial \mathbf{z}_t^L} \\ &= \frac{\partial \sum_{i=1}^K \tilde{y}_i \log \sum_{j=1}^K \exp \left(z_j^L \right)}{\partial \mathbf{z}_t^L} - \frac{\partial \sum_{i=1}^K \tilde{y}_i \log \left(\exp \left(z_i^L \right) \right)}{\partial \mathbf{z}_t^L} \\ &= \frac{\partial \log \sum_{i=1}^K \exp \left(z_j^L \right)}{\partial \mathbf{z}_t^L} - \frac{\partial \sum_{i=1}^K \tilde{y}_i z_i^L}{\partial \mathbf{z}_t^L} \\ &= \left(\mathbf{v}_t^L - \tilde{\mathbf{y}} \right) \quad \left[\text{also} = \nabla_{\mathbf{b}_t^L} Q_{CE}(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) \right]\end{aligned}$$

Backpropagation: Middle layers

$$\begin{aligned}\nabla_{\mathbf{W}_t^\ell} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) &= \nabla_{\mathbf{v}_t^\ell} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) \frac{\partial \mathbf{v}_t^\ell}{\partial \mathbf{W}_t^\ell} \\ &= \text{diag} \left(g' \left(\mathbf{z}_t^\ell \right) \right) \mathbf{e}_t^\ell \frac{\partial \left(\mathbf{W}_t^\ell \mathbf{v}_t^{\ell-1} + \mathbf{b}_t^\ell \right)}{\partial \mathbf{W}_t^\ell} \\ &= \text{diag} \left(g' \left(\mathbf{z}_t^\ell \right) \right) \mathbf{e}_t^\ell \left(\mathbf{v}_t^{\ell-1} \right)' \\ &= \left[g' \left(\mathbf{z}_t^\ell \right) \circ \mathbf{e}_t^\ell \right] \left(\mathbf{v}_t^{\ell-1} \right)'\end{aligned}$$

and

$$\nabla_{\mathbf{b}_t^\ell} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) = g' \left(\mathbf{z}_t^\ell \right) \circ \mathbf{e}_t^\ell,$$

where $\circ$ is the Hadamard (element-wise) product, $g'(\cdot)$ is the element-wise derivative of the activation function (see below), and $\mathbf{e}_t^\ell \equiv \nabla_{\mathbf{v}_t^\ell} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}})$ is the error at layer ℓ (see below). where the i th element of $\mathbf{v}_t^L$ is $\exp \left(z_i^L \right) / \sum_{i=1}^K \exp \left(z_j^L \right)$.

Activation functions

Activation function derivatives, $g'(\mathbf{z}_t^\ell)$:

- sigmoid: $g'(\mathbf{z}_t^\ell) = (1 - \mathbf{v}_t^\ell) \circ \mathbf{v}_t^\ell$
- tanh : $g'(\mathbf{z}_t^\ell) = 1 - \mathbf{v}_t^\ell \circ \mathbf{v}_t^\ell$
- ReLU : $g'(\mathbf{z}_t^\ell) = \max(0, \text{sgn}(\mathbf{z}_t^\ell))$, where $\text{sgn}(u)$ returns -1 if $u < 0$, 0 if $u = 0$, and $+1$ if $u > 0$; note, this is applied element-wise.

Error functions in backpropagation

$$\begin{aligned}\mathbf{e}_t^{L-1} &= \nabla_{\mathbf{v}_t^{L-1}} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) \\ &= \frac{\partial \mathbf{z}_t^L}{\partial \mathbf{v}_t^{L-1}} \nabla_{\mathbf{z}_t^L} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) \\ &= \frac{\partial (\mathbf{W}_t^L \mathbf{v}_t^{L-1} + \mathbf{b}_t^L)}{\partial \mathbf{v}_t^{L-1}} \mathbf{e}_t^L \\ &= (\mathbf{W}_t^L)' \mathbf{e}_t^L\end{aligned}$$

And, similarly, for $\ell < L$,

$$\mathbf{e}_t^{\ell-1} = (\mathbf{W}_t^\ell)' \left[g'(\mathbf{z}_t^\ell) \circ \mathbf{e}_t^\ell \right].$$

It is important to know that the error depends on the error from the level above it - that is, the error is propagating backwards (backpropagating)!

Backpropagation Algorithm for DNN

We start with M samples of $\mathbf{X}, \tilde{\mathbf{Y}}$

Init: Weights $\{\mathbf{W}_0^\ell, \mathbf{b}_0^\ell\}$, $0 < \ell \leq L$, $t = 0$; learning rate η ; minibatch size, M_b

```
1 while Stopping criterion not met do
2     select minibatch  $\mathbf{X}_{M_b}$  and  $\mathbf{Y}_{M_b}$  with  $M_b$  elements;
3     call DNN Forward Computation algorithm given  $\mathbf{X}_{M_b}$  and current  $\{\mathbf{W}^\ell, \mathbf{b}^\ell\}$ ;
4      $\mathbf{E}_t^L \leftarrow \mathbf{V}_t^L - \tilde{\mathbf{Y}}$  (note, each column of  $\mathbf{E}_t$  is  $\mathbf{e}_t^L$  for each minibatch sample);
5      $\mathbf{G}_t^L \leftarrow \mathbf{E}_t^L$ ;
6     for  $\ell \leftarrow L$  to 1 do
7          $\nabla \mathbf{W}_t^\ell \leftarrow \mathbf{G}_t^\ell (\mathbf{v}_t^{\ell-1})'$ ;
8          $\nabla \mathbf{b}_t^\ell \leftarrow \mathbf{G}_t^\ell \mathbf{1}_M$ ;
9          $\mathbf{W}_{t+1}^\ell \leftarrow \mathbf{W}_t^\ell - \frac{\eta}{M_b} \nabla \mathbf{W}_t^\ell$  (update  $\mathbf{W}$ );
10         $\mathbf{b}_{t+1}^\ell \leftarrow \mathbf{b}_t^\ell - \frac{\eta}{M_b} \nabla \mathbf{b}_t^\ell$  (update  $\mathbf{b}$ );
11         $\mathbf{E}_t^{\ell-1} \leftarrow (\mathbf{W}_t^\ell)' \mathbf{G}_t^\ell$  (error backpropagation);
12        if  $\ell > 1$  then
13             $\mathbf{G}_t^{\ell-1} \leftarrow g'(\mathbf{Z}_t^{\ell-1}) \circ \mathbf{E}_t^{\ell-1}$ 
14    end
15 end
```

This will output the DNN weights $\{\mathbf{W}^\ell, \mathbf{b}^\ell\}$, $0 < \ell \leq L$

Implementation

The backpropagation algorithm is really quite simple. The challenge is to implement it in a way that is practical and generalizable. Some of the important issues are briefly described below.

We need to preprocess the data, we have two primary choices:

- Feature normalization per sample: it sometimes helps to center each input $\mathbf{x}_m$, by subtracting off its mean (e.g., with an image)
- Global feature standardization: this is more common, and subtracts the average from all of the elements of each feature element across the entire training set, and dividing by the associated standard deviation (note, we would apply this to the test set as well, but using the means and standard deviations from the training set). Example: for images with pixel values between $[0, 255]$, we scale them to be between $[0, 1]$.

Implementation: Model Initialization

As we have seen, given the highly nonlinear nature of neural models, they can be very sensitive to the initial parameter values.

There are a couple general heuristic “principles” that have been suggested for a long time:

- one should choose parameters such that each hidden variable is operating in the linear range of its activation function; since this depends on both the weights and the inputs, it suggests why it is a good idea to standardize the inputs.
- one should choose the parameters randomly (this is due to the fact that the hidden units are interchangeable; one needs them to find different features, so it is important that they not have the same weights) However, this often is not enough for deep models.

Implementation: Regularization

There are two traditional norm-based regularization approaches to shrink the $\mathbf{W}$ parameters towards zero. Thus, the gradient term is:

$$\nabla_{\mathbf{W}_t^e} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}, \tilde{\mathbf{y}}) + \lambda \nabla_{\mathbf{W}_t^e} R(\mathbf{W})$$

where

- Ridge (L_2) Penalty: $\nabla_{\mathbf{W}_t^\ell} R(\mathbf{W}) = 2\mathbf{W}_t^\ell$
- Lasso (L_1) Penalty: $\nabla_{\mathbf{W}_t^\ell} R(\mathbf{W}) = \text{sgn}(\mathbf{W}_t^\ell)$

Implementation: Batch Normalization

Batch normalization is achieved through a normalization step that fixes the means and variances of each layer's inputs. Ideally, the normalization would be conducted over the entire training set, but to use this step jointly with stochastic optimization methods, it is impractical to use the global information. Thus, normalization is restrained to each mini-batch in the training process.

This scaling process happens *inside* the network architecture and therefore must be accounted for in backpropagation.

This mitigates *internal covariate shift*, in which each layer of a neural network has inputs with a corresponding distribution, which is affected during the training process by the randomness in the parameter initialization and the randomness in the input data.

Implementation: Dropout

Recently, dropout has become an important type of regularization in deep models. The idea is simple:

Randomly omit a certain (relatively small) percentage (say, α) of the hidden units in each hidden layer for each minibatch of samples during the backpropagation training. This helps to break up the dependence in the hidden units, and prevents overfitting (improving generalization). It is easy to implement dropout - one just sets the activation to 0 (no error propagates through that unit, and so the algorithm doesn't have to be changed in any other way). When applying test data, one can just discount all of the weights involved in the dropout by $(1 - \alpha)$ and treat it as a DNN without dropout. Alternatively, one can divide each activation by $(1 - \alpha)$ before the hidden units are dropped (so weights are automatically discounted and don't have to be adjusted for testing). Note, the same effect can occur if one deliberately adds noise to the training data. In this case, some of the neurons have to deal with the noise, limiting the fit, but increasing generalization.

Implementation: Early Stopping

Rarely does the best generalization of a deep model occur at the local minimum obtained by the gradient descent algorithm. Thus, an important form of regularization that facilitates generalization is called early stopping, which simply means one stops the gradient descent algorithm before it has “converged” (see the discussion in Goodfellow et al. 2016, Chap 7). The choice of the iteration to stop is essentially treated as a hyperparameter that must be chosen via a validation set. But, this is by far the simplest type of regularization one can do as it doesn’t require any change to the basic algorithm. It can be used (and typically is) with other forms of regularization (e.g., ridge, dropout, etc.).

Implementation: Weight Sharing

In some types of DNNs it makes sense that weights should be similar (e.g., imagine weights connected to components of an image or a group of words). One can force these weights to be identical, or can add a penalty term that favors them being similar.

For example, if one expects weights w_i and w_j to be similar, one could use a penalty $\lambda (w_i - w_j)^2 / 2$ in the objective function. Backpropagation algorithms can be modified easily to account for this type of penalty: e.g., when updating w_j , one would have the extra term $\eta \lambda (w_i - w_j)$, where η is the learning rate.

Implementation: Batch Size

As we have discussed, using batches in stochastic gradient descent is advantageous computationally (fewer samples but still embarrassingly parallel because of the batches). Some people suggest that it is preferable to have large gradient variance at the beginning of the optimization so the solution can jump out of local minima. This suggests that the minibatch sample should be smaller initially. We can then adjust it to have larger minibatches in later stages of the optimization (when we don't want to jump around as much). Thus, one can have the minibatch size vary through the iterations.

Implementation: Momentum

As we have discussed, it can be useful to include a “momentum” term in the gradient descent algorithm to take into account the previous gradients. That is, we take a convex combination of the previous gradient and the current gradient estimate: e.g.,

$$\Delta \mathbf{W}_t^\ell = \rho \Delta \mathbf{W}_{t-1}^\ell + (1 - \rho) \frac{1}{M_b} \sum_m \nabla_{\mathbf{W}_t^\ell} Q(\mathbf{W}, \mathbf{b}; \mathbf{x}_m, \tilde{\mathbf{y}}),$$

where ρ is the momentum factor. This value is typically chosen between 0.9 – 0.99

Note, if one is changing the minibatch size for each iteration, the ρ value isn't really valid (see the discussion in Yu and Deng, 2015, Chap 4).

Implementation: Learning Rate

Note that the actual learning rate is a function of both the “learning rate parameter” (η) and the minibatch size, M_b . Thus, one has to adjust the ratio $\eta^* = \eta/M_b$ to find a good learning rate. In situations where the minibatch size changes per iteration, then this clearly will change as well. There are heuristic ad hoc approaches to select this, but it seems to be very problem specific to me.

More formal mechanisms based on Bayesian optimization can be done in principle, but is often problematic for real-world problems.

Implementation: Network Architecture

The number of hidden units at each level is a parameter as well. Other than the fact that one needs enough hidden units to capture features (note, these are really just feature extraction models), and one doesn't want "too many" (to prevent overfitting), there isn't a whole lot of guidance.

In general, wide/shallow models are easier to overfit and narrow/deep models are easier to underfit. It is often recommended that the lower levels have more hidden units.

Section 3

Tensorflow

The Functional API

TensorFlow is an open-source machine learning library developed by Google that is widely used in industry and academia.

The *functional API* allows us to flexibly define a directed acyclic graph of layers and define how the data flows through the graph. Compared to the simpler *sequential* API, it allows for more complex models with

- multiple inputs and/or outputs
- shared layers
- models with branching architectures.

Many prebuilt layers, optimizers, and objective functions are included, but you can define your own as well.

Tensorflow in Practice

Using Tensorflow to fit deep learning models follows the following structure

- ➊ Define model architecture
 - Specify the layers of your network and how they are connected
 - ▶ activations as well
 - This includes the shapes of the input(s)/output(s)
 - No data is introduced at this stage!
- ➋ Compile model
 - Specify your loss function, optimizer details, stopping conditions, evaluation metrics
- ➌ Train model
 - The *first point* we introduce data
- ➍ Evaluate model

If it performs well, we may save it for later usage.

A Simple Deep Model

Generate simple data from

$$y = x^2 + \varepsilon$$

```
import tensorflow as tf
import numpy as np

# Generate random data
num_samples = 10_000
x_train = np.random.uniform(0,4.5,size=(num_samples, 1))
y_train = x_train**2 + np.random.normal(size=(num_samples, 1))

# Define the model architecture using the Functional API
inputs = tf.keras.Input(shape=(1,))
hidden1 = tf.keras.layers.Dense(32, activation='relu')(inputs)
hidden2 = tf.keras.layers.Dense(16, activation='relu')(hidden1)
outputs = tf.keras.layers.Dense(1)(hidden2)
model = tf.keras.Model(inputs=inputs, outputs=outputs)
```

A Simple Deep Model(cont)

```
# Compile the model
```

```
model.compile(optimizer='adam', loss='mse')
```

```
# Train the model
```

```
model.fit(x_train, y_train, epochs=10)
```

```
# Generate some test data
```

```
x_test = np.array([1, 2, 3, 4, 5, 6])
```

```
# Make predictions on the test data
```

```
y_pred = model.predict(x_test)
```

```
print(x_test**2 - y_pred[:,0])
```

```
## [-0.03872108, -0.04757214, -0.12748909, -0.23431015, 2.29092026, 7.63960
```

The model predict well within its range but doesnt extrapolate well.

Model Summary

The tensorflow model summary provides a concise overview of the architecture and properties of a DNN model. It includes the following information:

- Model type: The type of model, such as Sequential or Functional.
- Layer type: The type of each layer in the model, such as Conv2D, MaxPooling2D, or Dense.
- Output shape: The shape of the output tensor from each layer. This includes the batch size, as well as the height, width, and depth of the tensor.
- Number of parameters: The number of trainable and non-trainable parameters in the model.
- Parameter shapes: The shape of each parameter in the model.
- Trainable: Indicates whether each layer is trainable or not.
- Non-trainable params: The number of non-trainable parameters in the model.
- Total params: The total number of parameters in the model.

Model Summary

```
model.summary()
```

```
## Model: "model"
```

```
##
```

Layer (type)	Output Shape	Param #
--------------	--------------	---------

```
## =====
```

input_1 (InputLayer)	[(None, 1)]	0
----------------------	-------------	---

```
##
```

dense (Dense)	(None, 32)	64
---------------	------------	----

```
##
```

dense_1 (Dense)	(None, 16)	528
-----------------	------------	-----

```
##
```

dense_2 (Dense)	(None, 1)	17
-----------------	-----------	----

```
##
```

```
## =====
```

```
## Total params: 609
```

```
## Trainable params: 609
```

```
## Non-trainable params: 0
```

```
## -----
```

Dual output Model

Here we will simultaneously model

$$y_1 = x^2 + \varepsilon \quad \text{and} \quad y_2 = \frac{1}{2}x + \varepsilon$$

```
import tensorflow as tf
import numpy as np

# Generate random data
num_samples = 10_000
x_train = np.random.uniform(0,4.5,size=(num_samples, 1))
y1_train = x_train**2 + np.random.normal(size=(num_samples, 1))
y2_train = 0.5*x_train + np.random.normal(size=(num_samples, 1))
Y = np.hstack([y1_train, y2_train])

# Define the model architecture using the Functional API
inputs = tf.keras.Input(shape=(1,))
hidden1 = tf.keras.layers.Dense(32, activation='relu')(inputs)
hidden2 = tf.keras.layers.Dense(16, activation='relu')(hidden1)
outputs = tf.keras.layers.Dense(2, name='output')(hidden2)
dualmodel = tf.keras.Model(inputs=inputs, outputs=outputs)
```

Dual Output Cont

```
# compile the model with multiple losses
# Can also specify different losses e.g. ['mse', 'mae']
losses = {'output': 'mse'}
dualmodel.compile(optimizer='adam', loss=losses)

# Train the model
dualmodel.fit(x_train, Y, epochs=10)

x_test = np.array([1, 2, 3, 4, 5, 6])
dualmodel.predict(x_test)

## array([[ 1.0416288 ,  0.44393197],
##        [ 4.0403676 ,  0.9690831 ],
##        [ 9.029771  ,  1.4922793 ],
##        [15.981005   ,  1.9765208 ],
##        [22.83723    ,  2.4674568 ],
##        [28.338917   ,  3.0325086 ]], dtype=float32)
```

Dual outputs cont

Note that the previous example can have two quantitative outputs in one layer because they have the same activation function.

```
outputs = tf.keras.layers.Dense(2, name='output')(hidden2)
```

If one output were a single quantitative variable and one categorical variable with 10 levels, we would need to bifurcate the model after the last hidden layer as such.

```
reg_output = layers.Dense(1, name='reg_out')(hidden2)
class_output = layers.Dense(10, activation='softmax', name='class_out')(hidden2)
model = keras.Model(inputs=inputs, outputs=[reg_output, class_output])
```

Also, we should pass the named layers in at compile time to assign appropriate loss functions.

```
losses = {'reg_out': 'mse', 'class_out': 'sparse_categorical_crossentropy'}
loss_weights = {'regression_output': 0.5, 'classification_output': 0.5}
model.compile(optimizer='adam',
              loss=losses, loss_weights=loss_weights,
              metrics=['accuracy'])
```

Note: categorical_crossentropy loss is assuming a one hot encoded output whereas sparse_categorical_crossentropy assumes the output is $1, 2, \dots, K$

Dual input model

It is easy to have *branches* of a DNN that combine to create one output

```
import tensorflow as tf
from tensorflow.keras import layers

input1 = tf.keras.Input(shape=(10,), name='inp1_L0')
x1 = layers.Dense(64, activation='relu', name='inp1_L1')(input1)
x1 = layers.Dense(64, activation='relu', name='inp1_L2')(x1)

input2 = tf.keras.Input(shape=(5,), name='inp2_L0')
x2 = layers.Dense(32, activation='relu', name='inp2_L1')(input2)
x2 = layers.Dense(32, activation='relu', name='inp2_L2')(x2)

x = layers.concatenate([x1, x2])
output = layers.Dense(1, name='output')(x)
model = tf.keras.Model(inputs=[input1, input2], outputs=output)
```

This is typically done when one has multiple vastly different input types. For example we may want to present the network with an audio signal and a short time fourier transform of that signal as an image. This would require different branches for the different data structures.

Dual input model summary

```
## Model: "model_2"
```

```
##
```

Layer (type)	Output Shape	Param #	Connected to
inp1_L0 (InputLayer)	[(None, 10)]	0	[]
inp2_L0 (InputLayer)	[(None, 5)]	0	[]
inp1_L1 (Dense)	(None, 64)	704	['inp1_L0[0][0]
inp2_L1 (Dense)	(None, 32)	192	['inp2_L0[0][0]
inp1_L2 (Dense)	(None, 64)	4160	['inp1_L1[0][0]
inp2_L2 (Dense)	(None, 32)	1056	['inp2_L1[0][0]
concatenate (Concatenate)	(None, 96)	0	['inp1_L2[0][0] 'inp2_L2[0][0]
output (Dense)	(None, 1)	97	['concatenate[0]

```
##
```

```
## Total params: 6,209
```

```
## Trainable params: 6,209
```

```
## Non-trainable params: 0
```

Shared Layers

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# define the shared layer
shared_layer = layers.Dense(64, activation='relu')

# define the input layers
input1 = keras.Input(shape=(10,))
input2 = keras.Input(shape=(10,))

# apply the shared layer to the inputs
x1 = shared_layer(input1)
x2 = shared_layer(input2)

# define the separate output layers for each input
output1 = layers.Dense(1, name='output1')(x1)
output2 = layers.Dense(1, name='output2')(x2)
```

This approach creates two separate paths from the inputs that share the same weights. This allows the model to learn features that are relevant to both inputs. Useful for tasks where there are multiple sources of input data that share common features, such as in some natural language processing or image recognition tasks.

Other Layers: BatchNormalization

The BatchNormalization layer is a type of layer in that normalizes the inputs to each hidden layer. This helps to prevent the values in the network from getting too large or too small, which can cause problems with gradient propagation and slow down training.

```
inputs = keras.Input(shape=(10,))  
x = layers.Dense(64, activation='relu')(inputs)  
x = layers.BatchNormalization()(x)  
x = layers.Dense(32, activation='relu')(x)
```

- This technique is very often used when training very deep models.
- It also has a momentum parameter which controls the moving average of the mean and variance of the normalized inputs across batches.

Other Layers: Dropout

During training, the Dropout layer randomly drops out 50% of the inputs, which helps to prevent overfitting and improve generalization.

```
inputs = keras.Input(shape=(10,))  
x = layers.Dense(64, activation='relu')(inputs)  
x = layers.Dropout(0.5)(x)  
x = layers.Dense(32, activation='relu')(x)
```

- Higher values of rate lead to more aggressive regularization.
- The optimal value of rate changes on a case by case basis.

Note: During inference, the Dropout layer is disabled, and all inputs are passed through to the next layer. This can sometimes lead to the validation error rates being better than training error rates during monitoring.

L^2 Regularization in Dense layers

We can add L2 regularization to a TensorFlow model by using the `kernel_regularizer` parameter of a Dense layer.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers

...

x = layers.Dense(64, activation='relu',
                 kernel_regularizer=regularizers.l2(0.01))(x)
```

There is similarly an L1 regularizer.

Optimizer parameters

Modifying the learning rate and momentum of an optimizer can easily be done using the parameters available.

```
import tensorflow as tf
from tensorflow import keras

...

optimizer = keras.optimizers.SGD(learning_rate=0.001, momentum=0.9)
model.compile(optimizer=optimizer, loss=..., metrics=...)
```

One can explore the documentation for a complete list.

Learning rate scheduling

```
optimizer = keras.optimizers.SGD(learning_rate=0.001)

# define the learning rate schedule
def lr_schedule(epoch):
    lr = 0.001
    if epoch > 5:
        lr = 0.0001
    return lr

# define the learning rate callback
lr_callback = keras.callbacks.LearningRateScheduler(lr_schedule)

# compile the model with the optimizer
model.compile(optimizer=optimizer, loss='binary_crossentropy',
              metrics=['accuracy'])
history = model.fit(..., callbacks=[lr_callback])
```

Early Stopping

```
import tensorflow as tf
from tensorflow import keras

...

early_stopping_callback = keras.callbacks.EarlyStopping(monitor='val_loss',
                                                         patience=3)

history = model.fit(..., callbacks=[early_stopping_callback])
```

- To use early stopping, we define an EarlyStopping callback with a monitor parameter set to 'val_loss', which specifies the validation loss as the quantity to monitor. The patience parameter is set to 3, which means that training will stop if the validation loss does not improve for 3 consecutive epochs.
- During training, the EarlyStopping callback monitors the validation loss after each epoch, and stops training if the validation loss does not improve for the specified number of epochs.